An IDM alternative that can ship

Review and revised development plan

15 October 2026 | Planning only

Verdict

The supplied plan is a strong technical specification and a weak first-release contract. Its treatment of HTTP correctness, interrupted downloads, credentials, media quality and dependency provenance is substantially better than a typical feature checklist. Keep that foundation. The largest improvement is to turn its broad scope into independently useful releases, with early experiments that resolve the hardest uncertainties.

The recommended approach is **a reliable Windows download manager first, a qualified video edition second, and advanced IDM-like workflows afterward**. Keep the Rust/libcurl/Tauri direction as the baseline. Do not replace the whole architecture merely to make the plan look different.

What is already good

- The engine survives closing the window, owns job state, and keeps transfer bytes outside the browser messaging channel.
- Resume checks resource identity instead of assuming that a matching filename or size means matching content.
- The plan distinguishes real source quality from upscaling, and remuxing from transcoding.
- Browser limitations, encrypted media, dependency licences and store distribution are treated honestly.
- Its fault tests, feature identifiers and explicit exclusions provide a useful foundation for acceptance testing.

What needs to change

The original makes all 32 features part of the completed release, including a crawler, synchronization, FTP, enterprise authentication, ZIP parsing and several media paths. That couples basic usefulness to many independent projects. Packaging and browser feasibility also arrive too late in the sequence to inform architecture cheaply.

Several correct principles need more concrete contracts: identity must hold during every parallel request; browser handoff needs idempotency and cancellation recovery; durable completion needs a recoverable publication step; and extractor metadata needs an explicit boundary with the custom transport.

Recommended decision

Approve the direction as a roadmap, but replace the single comprehensive release commitment with the release gates in this document. Retain the original as a reference, not as an instruction to start coding. No application implementation or performance testing has been performed for this review.

Reading guide: pages 2-3 explain the changes and scope; pages 4-9 specify the design; pages 10-13 cover delivery, verification and decisions; pages 14-15 preserve feature traceability; page 16 lists checked sources.

1. Review findings, in priority order

Priority	Original location	Finding and recommended change
High	Sections 2 and 10	All F01-F32 features are release obligations. Split them into Core, Media and Extended releases; publish each release's actual coverage.
High	Sections 6 and 10	Browser work begins after the engine and UI. Prove native-host installation, extension permissions and replay limits in the first feasibility stage.
High	Section 6, file handoff	A durable queue acknowledgement does not prove the native transfer will succeed. Add a transaction ID, readiness state, expiration, cancellation acknowledgement and recovery rules.
High	Sections 5.1-5.3	Identity validation is sound but underspecified during concurrent transfer. Bind every accepted extent to one representation generation and conditional-request policy.
High	Section 7	yt-dlp discovery plus a custom HLS/DASH transport is significant integration work. Qualify a small manifest subset and a small site list before promising breadth.
Medium	Sections 5.3 and 11	Flush-before-journal is necessary, but publication, database configuration and recovery after rename need exact rules. Distinguish process crashes from actual storage power-loss qualification.
Medium	Sections 3 and 9	Installer, native libraries, media builds and updates have late integration risk. Build an installable skeleton early and verify the exact packaged binaries.
Medium	Sections 5.2 and 11	Worker counts, block sizes and timing thresholds are hypotheses. Start conservatively and qualify them against single-stream baselines before enabling adaptation.

Important qualifications

These are improvements to sequencing and precision, not evidence that the original is technically unsound. It already recognizes most underlying risks. A replacement that simply says “use yt-dlp and FFmpeg” would be less complete.

The original's avoidance of GPL dependencies and mandatory fees are product assumptions inherited from that document. They are retained as defaults here, rather than treated as new requirements from the review request. A different distribution model could justify a different engine choice.

The original also correctly avoids unsupported calendar promises. This revision adds provisional effort ranges for budgeting, with a mandatory re-estimate after feasibility work. They are judgments, not measured delivery commitments.

2. Product scope and release boundaries

Core release: dependable everyday downloads

Support Windows 11 x64, local NTFS destinations, HTTP/HTTPS, adaptive ranges where safe, pause/resume, interruption recovery, basic queues, priorities, limits, history, filename handling, one-time and recurring schedules while the user is logged in, and explicit browser-to-app downloads. Include an installer, attachment-origin handling, keyboard access and a tested update path.

Use Edge as the first browser qualification target. Add Firefox to the Core release only when its separate installation and handoff gates pass; otherwise publish it as the next supported integration. Chrome packaging is a distribution decision, not an assumption that developer mode is a consumer installer.

Core success: a user can install, send a supported file to the app, pause, restart Windows, resume safely and find the completed file. Unsupported transfers remain in the browser with an explanation.

Media release: preserve accessible source quality

Add direct media, a documented clear HLS VOD subset, a documented static DASH subset, exact video/audio selection, compatible stream-copy assembly, expired-link recovery where identity is provable, and a small named set of qualified extractors. Test genuine 7680 x 4320 media, separate audio, supported HDR metadata and merge failure recovery.

If video is the essential launch differentiator, launch after Core plus Media. Keep the same technical gates; do not quietly relabel Core as delivering the full video goal.

Extended releases: selected advanced workflows

Add automatic capture, broader session integration, multiple advanced queues, quotas, HTTP/SOCKS proxies, enterprise authentication, FTP/FTPS, grabber/sync, ZIP preview, wake scheduling, scanner adapters and additional languages according to demonstrated demand. Each addition gets its own compatibility statement and tests.

Boundaries that remain

Use original code, name and visual assets. Require no hosted backend for normal operation. Do not promise every website, universal acceleration, protected-media acquisition or a perfect offline copy of a dynamic application. Exclude DRM circumvention, TLS interception, automatic execution of downloaded files, transcoding, HTTP/3, torrents and cross-platform support from these releases.

Scope control: each feature has a release, acceptance evidence and an owner. A deferred feature is recorded as deferred, never counted as implemented. The complete mapping back to F01-F32 is on pages 14-15.

3. Architecture and dependency decisions

Retain one per-user Rust engine, SQLite, a dedicated libcurl multi worker, and a Tauri/React/TypeScript UI. Keep one repository with clear module boundaries; separate services are unnecessary. The browser native host forwards bounded commands to an authenticated local engine connection. Media workers are optional child processes added in the Media release.

Component	Ownership and boundary
Engine	Job state, schedules, retry policy, budgets and credentials. Sole authority for state transitions.
Transport	HTTP operations and response validation. Handles remain on the owning transport thread.
Storage	Extent leases, positional writes, checkpoints, recovery and final publication.
Desktop UI	Original, accessible management interface. No raw secrets or remote executable pages.
Browser host	Versioned, bounded messages; browser/profile context; idempotent handoff.
Media workers	Constrained discovery and local assembly. No independent bulk downloads by default.

Do not overbuild the initial dependency set

Pin only dependencies used by the current milestone. Defer Python, yt-dlp, FFmpeg, JavaScript runtimes, HTML/CSS parsers and ZIP parsing until their release needs them. Preserve exact build configurations, hashes and licences for all shipped components, including native transitive dependencies.

Prove the packaged libcurl build's TLS, HTTP/2, proxy and authentication capabilities rather than inferring them from the library's general feature list. Add those features when needed. Tokio may coordinate tasks, but does not turn a blocking transport API into an asynchronous one.

Build versus reuse is a decision, not a slogan

The custom engine remains the baseline because the supplied plan seeks tight control and a permissive dependency policy. If GPL-compatible distribution is acceptable, evaluate aria2 in a short alternative-design exercise: it already offers multi-connection transfers and RPC. Its upstream licence is GPL-2.0-or-later. This may reduce custom transport work, but does not solve browser integration, media discovery or product UX. It also needs its own security and recovery qualification. R10

Do not assume that a subprocess boundary decides the licensing of an entire combined product. Record the distribution choice before switching engines. A Windows-native UI is another possible tradeoff if the team lacks web/Rust experience; there is insufficient evidence to recommend a stack rewrite now.

4. Transfer correctness before acceleration

Define one representation generation

Associate each job generation with its resource, selected representation, stable request context and validator policy. Never use an ETag as a global content identifier across unrelated URLs. A refreshed URL must establish identity through the qualified source adapter or a trusted expected digest; otherwise restart in a new generation.

Use a bounded range GET only for sources known to be replayable. For an unknown or single-use source, offer a single streaming request without a preliminary probe; retries may require a fresh link. A successful probe cannot guarantee that every later request will work.

Adopt a conservative policy

Observed condition	Planned behavior
Valid ranges, known length, qualified identity	Permit segmented transfer within one generation.
Ranges ignored or total length unknown	Use a sequential path; do not write a full response at a range offset.
Missing or weak validator	Core defaults to sequential transfer and restart after interruption. Later exceptions need a documented identity mechanism.
Resource changed or identity uncertain	Stop workers, fence old writes and create a new generation. Never offer unsafe splicing as an ordinary resume option.
Expired or non-replayable request	Request a fresh authorized link or preserve the browser workflow.

HTTP ranges and conditional headers must follow RFC 9110. Weak ETags are unsuitable for If-Range; dates are usable there only under the standard's strong-validator conditions. An If-Range mismatch can produce a full response. The implementation must treat that as a mode/identity transition. R1

Require validated headers before passing data to the file writer. Every range response must fit its lease and generation. The later implementation should choose one consistent If-Match or If-Range strategy, with explicit behavior for precondition failure, ignored ranges and invalid lengths.

Keep byte accounting unambiguous

Request identity encoding for segmented transfers and disable automatic content decoding on that path. libcurl's encoding option can enable decompression, so configuring the request header alone is not the full design. Reject unexpected encoded range responses from segmentation; a separate sequential path may handle decoding with honest size reporting. R2

Only after these rules pass tests should the engine try additional workers. Start with one stream; qualify two and then four ranges. Treat the original 4 MiB blocks, 3-second windows and eight-worker cap as tuning candidates, not requirements.

5. Storage, recovery and completion

Separate received bytes from durable bytes

Use disjoint extent leases and a generation number. Cancellation or replacement invalidates leases; late callbacks cannot write to a new generation. Display downloaded progress separately from the durable checkpoint where useful. “Paused” means workers are stopped and the promised checkpoint has completed.

For each checkpoint: finish writes for selected extents, flush the data file successfully, then commit those extents and their generation to SQLite. A failed flush never produces committed progress. Begin with WAL plus synchronous=FULL and a single database writer; benchmark batching before weakening durability. SQLite documents that WAL with NORMAL can lose recent commits after power loss. R5

Record compact extent metadata, not a transaction for every network buffer. Re-download uncommitted extents after a crash. Use checksums for recovery where they add evidence; locally generated hashes detect local changes but do not authenticate a publisher.

Make final publication recoverable

1. Stop writes and verify completeness, expected size and any supplied trusted digest.
2. Persist a publication intent containing the job generation, owned staging-file identity and reserved final destination.
3. Apply required attachment-origin metadata and finish configured trust checks. Preserve an explicit blocked or scan-failed state.
4. Rename within the qualified filesystem without silently overwriting a user's file. Confirm that the final file is the expected owned file.
5. Commit completion only after publication succeeds. Retain enough intent data to reconcile a crash between rename and the database commit.

At startup, reconcile intent plus file identity. A matching final file can complete the transaction; a remaining staging file can retry publication. A missing, ambiguous or unrelated file produces recovery attention, not a false “completed” result. Antivirus quarantine after completion updates availability without rewriting historical transfer success.

State the durability limit precisely

Qualify local NTFS first. Removable drives, network shares, cloud-synchronized folders and other filesystems need separate tests. Do not claim that process-kill tests establish power-loss safety. Windows exposes flushing controls, but storage hardware caching is also relevant. R6 R7

Budget disk space for partial files, media tracks, assembled output and recovery retention across all jobs. Sparse allocation is not a reservation of future free space. Never remove an existing final file merely to make room for an unfinished replacement.

6. Browser integration with a safe handoff

Ship an explicit action first

Start with “Download with this app” on a supported link or selected page candidate. This avoids canceling a browser transfer that has already begun. It also provides a clear user action for permission and destination decisions. Build native-host installation and absence/error handling during feasibility work.

The downloads API can monitor, pause and cancel browser jobs; those operations do not form an atomic transaction with a native engine. Most Manifest V3 extensions also cannot use webRequestBlocking. Therefore automatic takeover requires a restricted eligibility policy and its own recovery protocol. R3 R4

Protocol for later automatic takeover

1. **Identify.** Create a stable handoff ID scoped to the extension installation/profile and browser download. Persist it on both sides; retries return the same native job.
2. **Prepare.** Restrict takeover to qualified replayable requests. Pause the browser job if supported and verify its actual state. If it already completed, keep that result and abandon native takeover.
3. **Validate.** The native engine checks destination, request context and access, then persists readiness. A queue acknowledgement alone is not readiness. Hold any native probe result within its lease; it is still not a promise of eventual success.
4. **Commit.** Cancel the browser job, confirm the resulting state, and commit native ownership with the same handoff ID. On timeout, query both sides before doing anything again.
5. **Recover.** If readiness fails before cancellation, resume the browser job. After confirmed cancellation, use the recorded native job or explain how to obtain a fresh link; never promise that the canceled browser job can always be resurrected.

An expired preparation lease discards only app-owned staging work. Extension restart, native-host restart and duplicate messages must not create extra jobs or cancellation loops. If ambiguity remains, preserve the browser result and stop automatic takeover.

Request context and private sessions

POST exports, blobs, uncertain one-use links, non-transferable sessions and unsupported partition/container contexts stay in the browser. A native client does not automatically reproduce browser TLS, anti-bot or client-certificate behavior. Decline unsupported cases instead of copying broad profiles.

Keep site-scoped credentials separate from display URLs, recheck redirects, and avoid implicit private-network access from web pages. Disable private-browsing integration by default; if added later, require an explicit session mode with no normal history or persisted credentials.

Native messaging carries bounded control messages only; Chrome documents a 1 MB host-to-browser limit. The original 256 KiB application limit is a reasonable conservative choice. R8

7. Video as a qualified second product layer

Make discovery and transfer a tested contract

Use yt-dlp only through a pinned wrapper that converts output into a versioned internal plan: source identity, exact format, protocol, URLs, required headers, expiry, tracks and protection status. Discovery-only extraction exists upstream, but its returned objects are not guaranteed to be directly JSON serializable. Normalize and validate them; never persist arbitrary extractor output as an engine contract. R9

Not every extractor result is necessarily transferable by the custom engine. A site is enabled only after the exact extraction-to-transfer path passes. If it requires an unsupported downloader or browser-bound session, report that limitation. Add worker-managed bulk transfer only after proving shared scheduling, accounting and pause/recovery behavior.

Introduce formats in this order

Stage	Supported subset and gate
M1	Direct media and separate direct audio/video tracks. Correct stream selection and local remux.
M2	Clear HLS VOD: a fixed rendition, one selected audio track, supported MPEG-TS/fMP4 forms, initialization and byte ranges. Reject unsupported timeline changes.
M3	Static DASH: one period, supported SegmentTemplate/Timeline or SegmentList forms, selected representations and local assembly. Other addressing forms wait for fixtures.
M4	Small named extractor list, expired-link refresh, additional tested languages/formats and playlists.

This deliberately narrows the original first media release. Add discontinuities, multi-period DASH and live recording only after independent qualification. HLS manifests describe more than a list of interchangeable files; sequence, initialization and discontinuity semantics matter. R11

Reject encrypted media in the initial policy, including clear-key/AES-encrypted variants; encryption is not synonymous with DRM, but broad key handling is outside this scope. HTTPS remains supported. Test protection changes on refreshed manifests as well as initial discovery.

Preserve the selected source

No silent quality downgrade, skipped fragments or forced MP4 conversion. Select compatible containers, keep tracks until verification succeeds, and inspect dimensions, duration, audio, timing and supported HDR metadata. Stream-copy avoids decoding/re-encoding but does not make the output container byte-identical. R12

Use actual 8K fixtures; resolution is metadata, not a different transfer algorithm. Packet comparisons must account for legitimate bitstream/container transformations. Keep a tested reference playback check alongside structural verification. Do not promise universal device playback or every form of HDR metadata preservation.

8. Resource limits and security boundaries

One scheduler for product promises

Use bounded concurrency and buffers across jobs, tracks and origins. Separate fairness from speed tuning: a fast job must not starve another queue. Shrink concurrency on throttling and sustained regressions, respect Retry-After, and avoid tuning from one noisy sample.

Rate limits apply at global, queue and job levels. Record useful bytes and transferred bytes separately. Retries count toward transferred bytes. Reserve a bounded allowance for in-flight work and disclose that application byte counters differ from network-provider billing.

Do not describe limits as exact if extractor discovery traffic bypasses the accounting path. Either meter it through a qualified worker interface or clearly scope the limit to managed transfer traffic and bound discovery separately. Start with speed caps; defer billing-style calendar quotas until accounting has been validated.

Backpressure must cover libcurl's own buffering as well as app buffers. Its pause behavior can redeliver callback data and deliver buffered bytes immediately on resume; that needs careful accounting and memory tests, especially with multiplexing. R13

Protect both desktop and helper boundaries

- Restrict engine IPC to the current user, reject remote pipe clients and version every request. Extension allowlists constrain browser use; they do not isolate the app from malware already running as that Windows user.
- Keep credentials and signed URL tokens out of arguments, logs, notifications, normal history exports and crash diagnostics. Store persistent secrets only by explicit choice using Windows protection facilities.
- Use fixed helper paths, bounded inputs and clean environments. Child-process limits control resource use; they are not a sandbox. For the media release, qualify a restricted execution boundary and a job-specific staging directory.
- Build FFmpeg without network protocols and allow only necessary local formats. Validate every path, include and local manifest entry; a file-only helper can still access unintended local files if paths are not constrained.
- Sanitize filenames, reserved Windows names, alternate data stream syntax and traversal. Reject ambiguous reparse-point escapes from app-owned staging areas.

Plan for support without collecting secrets

Offer a user-reviewed diagnostic export containing app/component versions, redacted events, error categories and job IDs. No raw cookie jars, request headers or tokenized URLs. Keep telemetry optional, with no account requirement for ordinary operation.

9. Delivery sequence and effort envelope

These are provisional engineering estimates for experienced contributors, including ordinary testing and rework. An engineer-week is effort, not elapsed time. Store review, procurement, legal review and unknown site changes are excluded. Re-estimate after G0; do not convert these ranges into a promised completion date.

Gate	Deliverable and exit evidence	Effort
G0: feasibility	Installable shell; native host round trip; packaged libcurl capabilities; interrupted-download fixture; direct audio/video remux experiment; written architecture decisions.	2-4 engineer-weeks
G1: transfer	Sequential and conditional ranges, writer, journal, publication recovery, retry and limits. Corruption/failure suite passes.	6-10
G2: usable Core	Desktop workflows, schedules, explicit browser handoff, accessibility, attachment-origin handling and history. Real restart workflows pass.	4-7
G3: Core qualification	Clean-machine installation, updates/migration recovery, notices, diagnostics, performance report and supported-browser matrix.	3-5
G4: Media	M1-M4 in sequence, supported-site list, 8K/audio verification, helper hardening and packaged-source compliance.	Additional 8-14
G5: Extended	Select advanced workflows individually. Define estimates and gates per feature.	Re-estimate by scope

Budgeting envelope: Core is roughly 15-26 engineer-weeks under these assumptions; Core plus Media is roughly 23-40. Low end assumes reusable expertise and few platform surprises. Broad site compatibility or full original feature coverage can take substantially longer. More contributors do not divide the schedule linearly because transfer and recovery correctness constrain dependent work.

Stop conditions prevent expensive detours

If automatic browser capture cannot be made reliable, ship explicit actions. If a site cannot use the supported transport, leave it disabled. If a minimal FFmpeg build cannot preserve required metadata, qualify another acceptable build or narrow the compatibility claim. If acceleration does not beat a single stream, retain the single stream for that class of source.

Ownership and maintenance

Assign accountable owners for engine/storage, desktop/browser, media, and release/security even if one person fills several roles. Reserve capacity after launch for browser changes, dependency vulnerabilities and extractor regressions. Review compatibility monthly and triage severe security issues promptly. These are future operating requirements, not scheduled tasks created by this document.

10. Acceptance evidence and performance targets

Retain the original regression matrix, but run a release-specific subset at every gate. Use deterministic local/LAN fixtures first. Real-site checks are supplementary, dated and restricted to permitted test content. Every release report maps a claim to a fixture, result and known limitation.

Area	Required evidence before claiming support
Bytes and identity	Expected digest matches; changed resource during parallel requests cannot mix generations; incorrect ranges, encoded responses and weak validators choose safe behavior.
Recovery	Fault injection before/after write, flush, journal, publication intent, rename and completion. Repeat selected tests in a disposable VM with OS interruption. Label the limits of storage/power-loss testing.
Storage	More-than-4-GiB files, disk exhaustion, permission changes, collisions, antivirus locks and quarantine, Unicode/long paths, abandoned staging cleanup.
Browser	Missing app, version mismatch, denied permission, worker restart, duplicate handoff, timeout at every takeover step, browser already complete and non-replayable fallback.
Media	Separate audio, genuine 8K, selected language, supported HDR, missing fragment, changed manifest, expired link and failed assembly. No silent downgrade or partial success.
Security	Secret redaction; redirect and private-network policy; malformed input; parser limits; helper path confinement; no unauthenticated local listener.
Release	Fresh non-admin machine, upgrade with paused jobs, failed migration, signature tampering, old-version replay, native-host install/uninstall and WebView2 prerequisites.

Proposed targets, not results

Zero mismatched outputs in the deterministic correctness suite is a release gate, not proof against all possible corruption. Record seeds and replay failed cases. Fuzz range/manifest/message parsing and exercise state transitions, rather than merely testing successful downloads.

Compare the new engine with a single libcurl stream under identical conditions: connection-limited server, aggregate-limited server, high latency, normal unconstrained transfer and slow disk. Use at least five repetitions per case and record median time, spread, retries, CPU, resident memory and connection count.

Initial targets are at least 90% of the measured attainable throughput on suitable fixtures and no more than 10% median slowdown against an already-saturated single stream after adaptation. Investigate failures before changing targets. Measure buffers, libcurl/native allocations, UI and helper memory separately; the original 64 MiB buffer proposal is not a whole-process memory guarantee.

Define UI performance separately: interactive pause/cancel acknowledgement, a responsive large history and bounded event update frequency under full transfer load. Benchmark on named hardware; do not invent universal timing claims.

11. Installation, updates and distribution

Packaging starts at G0

Produce a per-user installer and clean-machine smoke test before the engine grows. Install only app-owned files and explicit integration entries. Ordinary downloads stay unprivileged. Preserve downloads on uninstall and preserve pending jobs across upgrades. Qualify WebView2 detection and the supported runtime installation route.

Keep the original distinction between open-source licence fees and operating costs. Store accounts, installer trust, release hosting, bandwidth and maintenance are separate budget decisions. Do not promise unlimited free distribution or ask users to disable Windows protections.

Edge currently documents no registration fee; Chrome requires a one-time registration fee; Firefox has its own signing/distribution rules. Recheck these at release time. Packaging feasibility and store acceptance are separate gates. R14 R15 R16

Signatures do not implement recovery policy

Tauri verifies signatures on update artifacts. Its documentation does not provide the product's database-migration rollback policy, release freshness policy or signing-key recovery process. Build those explicitly. Its update metadata includes an artifact signature; do not assume every metadata field is separately signed. R17

Use HTTPS, verify artifacts, and check an authenticated package identity/version before installation. Persist the highest accepted release sequence. Reject ordinary replay of an older release. Emergency recovery should use a newly signed recovery release with a higher sequence, rather than casually disabling version protection.

Before migration, pause/checkpoint jobs, close database writers and create a validated backup including the required SQLite state. Pair a compatible application version with its database state during recovery. Restoring an old executable against a newer incompatible database is not rollback. Document recovery of partial files separately from database recovery.

Keep shipped code auditable

Generate a release SBOM covering Rust, JavaScript, native libraries, Python, helpers and installer components. Inspect the exact binaries and archive hashes, build recipes, notices and corresponding source where required.

The yt-dlp project distinguishes its Unlicense source/wheel from PyInstaller executables containing GPLv3+ components. FFmpeg licensing depends on enabled components. Keep the reviewed private-runtime and controlled-helper approach if the original dependency policy is retained. R9 R18

Media updates may have a faster release cadence, but use the same verification rules, a compatibility version range and a tested rollback. No arbitrary extractor/plugin auto-update channel.

12. Decisions to record before future implementation

These decisions can be resolved during G0. The defaults below make the proposal concrete without treating assumptions in the supplied document as permanent requirements.

Decision	Recommended default	Revisit when
Product promise	Reliable downloads plus a qualified media release.	Research shows another workflow is the primary reason to use it.
Release timing	Ship Core independently; wait for Media if video is essential to launch.	Launch audience is defined.
Engine	Rust/libcurl with a custom scheduler and journal.	GPL-compatible reuse or team experience changes the economics.
Initial browser	Edge first; Firefox after its own qualification.	The intended audience primarily uses another browser.
Initial storage	Local NTFS; explicit unsupported/limited status elsewhere.	Removable or network destinations are a launch requirement.
Initial media	Direct files, restricted clear VOD subsets and a named extractor list.	Additional formats have reproducible fixtures and maintenance ownership.
Costs	No mandatory paid runtime service; distribution budget decided separately.	Public signing, hosting or store presence is required.
Privacy	Local operation, no account, optional diagnostics.	A specific cloud workflow is requested.

Required artifacts at each milestone

Maintain a short decision log, dependency lockfiles and build recipes, protocol/IPC schemas, fixture catalogue, test report, known-limitations list and feature mapping. An implementation handoff should describe observed behavior and reproducible evidence, rather than only marking checklist items complete.

For Core, provide installation and recovery instructions plus a clear browser fallback guide. For Media, add exact format/site coverage and the date each live path was checked. For Extended, update the same matrix so that expansion does not erase earlier boundaries.

Why this is a better plan

It retains the original's strongest engineering safeguards while reducing the number of unknowns that must succeed before a user gets value. Browser distribution, durable storage and media feasibility are tested early. Every shipped feature has a smaller, explicit contract; later features no longer hold basic download management hostage.

The result is a plausible development program with useful intermediate releases. It is still a substantial desktop product with ongoing compatibility work, not a short wrapper around a command-line downloader.

Appendix A. Feature traceability: F01-F16

“Core,” “Media” and “Extended” refer to the proposed releases. A split row preserves the full original goal while assigning its parts to different releases. Extended items remain roadmap scope and do not block Core or Media.

ID	Original capability	Proposed release and boundary
F01	Acceleration	Core: safe adaptive ranges; sequential fallback.
F02	Pause/resume/retry	Core: qualified identity and bounded retry.
F03	Crash/power-loss recovery	Core: tested journal and publication recovery; bounded durability claim.
F04	Queues/priorities	Core: basic queues and fairness. Extended: advanced named-queue policies.
F05	Scheduling	Core: logged-in schedules and missed-run policy. Extended: wake and advanced actions.
F06	Bandwidth/usage limits	Core: speed caps. Extended: rolling/calendar quotas and metered-network policies.
F07	Categories/folders	Core: destination and basic category rules. Extended: richer rules.
F08	Browser handoff	Core: explicit supported GET action. Extended: qualified automatic takeover.
F09	Browser video button	Media: permitted, verified candidates on qualified pages.
F10	Highest-quality video	Media: direct media, tested clear VOD and named extractors, including actual available 8K.
F11	Audio/video combination	Media: tested stream-copy/container combinations.
F12	Batches/multiple videos	Core: file URL lists, import/export and deduplication. Media: video batches and qualified playlists.
F13	All page links	Extended: bounded discovery, preview and selection.
F14	Website grabber	Extended: bounded crawl and static copies; separate project gate.
F15	Periodic synchronization	Extended: staged replacement and version preservation.
F16	HTTP/HTTPS	Core: qualified redirects, TLS, IPv4/IPv6, HTTP/1.1 and HTTP/2.

Scope change to notice: automatic interception, page-link collection and advanced quota rules move later. Video remains a named release with a genuine quality-preservation goal. These are deliberate scheduling changes, not claims of full IDM parity.

Appendix A. Feature traceability: F17-F32

ID	Original capability	Proposed release and boundary
F17	FTP/FTPS	Extended: qualified retrieval and restart behavior.
F18	Proxies	Extended: HTTP CONNECT/SOCKS and explicit credential/DNS rules.
F19	Authentication	Extended: Basic/Digest and qualified SSPI/NTLM/Kerberos.
F20	Signed-in downloads	Core: only qualified explicit site context. Media/Extended: broader tested contexts.
F21	Expired-link refresh	Media: source/format identity proof. Extended: general file-link workflow.
F22	Antivirus integration	Core: attachment-origin handling and honest status. Extended: installed-scanner adapters.
F23	ZIP preview	Extended: bounded ordinary/ZIP64 listing. Prefer a reviewed parser; justify custom parsing if needed.
F24	Clipboard/drag/CLI	Core: paste, drag links and minimal CLI. Extended: opt-in monitoring, drag files out and integration polish.
F25	Progress/history	Core: searchable history, speed/ETA and actionable reasons.
F26	Customizable interface	Core: keyboard, tray, themes and basic columns. Extended: richer toolbar customization.
F27	Languages/accessibility	Core: English, accessible controls and localization foundation. Extended: reviewed Hindi/Gujarati and RTL qualification.
F28	Installation/updates	Core: per-user installation, signed updates, migration recovery and uninstall. Skeleton starts at G0.
F29	Completion actions	Core: notify/reveal/exit. Extended: opt-in sleep/shutdown countdown.
F30	Large files/integrity	Core: 64-bit counters, disk budgeting and trusted-digest validation when supplied.
F31	Sleep/network recovery	Core: conservative resume/revalidation. Extended: broader session renewal cases.
F32	Other app integration	Core: minimal CLI. Extended: supported local IPC for cooperating apps.

No row is considered satisfied merely because a library supports it upstream. Implemented behavior, packaged capabilities and acceptance evidence determine support. A claimed platform limitation must identify the affected workflow; it cannot serve as a blanket substitute for an unfinished feature.

Appendix B. Sources and evidence

The review evaluated the supplied **IDM-Alternative-Implementation-Plan.md**, dated 13 October 2026, including its feature matrix and sections 1-14. External checks below were made on 15 October 2026. They support specific platform facts; the proposed release structure, effort estimates and engineering defaults are recommendations. This was a targeted feasibility review, not a full dependency or legal audit.

- **R1.** RFC 9110, HTTP Semantics: conditional requests, validators and ranges, particularly sections 8.8, 13 and 14.
- **R2.** libcurl: CURLOPT_ACCEPT_ENCODING: encoding negotiation and automatic decoding.
- **R3.** Chrome downloads API: download states and pause/cancel controls.
- **R4.** Chrome webRequest API: Manifest V3 blocking and host-permission limitations.
- **R5.** SQLite synchronous pragma: WAL durability settings and power-loss distinctions.
- **R6.** Microsoft FlushFileBuffers: Windows flushing API.
- **R7.** Microsoft file buffering: system and hardware caching considerations.
- **R8.** Chrome native messaging: host lifecycle, message framing and size limits.
- **R9.** yt-dlp README: extraction interface, dependencies and distribution-specific licensing.
- **R10.** aria2 official project: transfer features, RPC and GPL-2.0-or-later licence.
- **R11.** RFC 8216, HTTP Live Streaming: playlist, segment and timeline semantics.
- **R12.** FFmpeg streamcopy documentation: stream-copy behavior and constraints.
- **R13.** libcurl: curl_easy_pause: pause/resume callbacks and buffering caveats.
- **R14.** Microsoft Edge developer registration: registration and fee policy.
- **R15.** Chrome Web Store developer registration: registration fee requirement.
- **R16.** Firefox signing and distribution: supported distribution and review framework.
- **R17.** Tauri updater: artifact signing, update metadata and version comparison.
- **R18.** FFmpeg licensing: licence effects of optional components.

At implementation time, pin exact versions and recheck changing upstream documentation, store policies and site compatibility. No unchanged source link establishes that a future packaged product meets these requirements.